

Title: Implementation of Merge Sort using Sequential and Parallel Processing

Name: Aryamann Srivastava

Course: B.E CSE

Subject: IBP

Date: 19/04/2026

1. Introduction

Merge Sort is a divide-and-conquer sorting algorithm with time complexity **$O(n \log n)$** . It works by dividing the array into smaller subarrays, sorting them, and merging them back.

In this project, we implement:

- Sequential (Normal) Merge Sort
- Parallel Merge Sort using multiprocessing

2. Objective

- To implement merge sort algorithm
- To compare performance of sequential and parallel approaches
- To analyze the impact of input size on execution time

3. Methodology

3.1 Sequential Merge Sort

- Recursively divide array into two halves
- Sort each half
- Merge sorted halves

3.2 Parallel Merge Sort

- Split array into two parts
- Use multiprocessing to sort both parts simultaneously
- Merge the results

4. Experimental Setup

- Language: Python
- Library used: multiprocessing

- Input sizes tested:
 - 10,000 elements
 - 1,000,000 elements

5. Results

Case 1: Small Input (10,000 elements)

- Normal Merge Sort Time: **0.016 sec**
- Parallel Merge Sort Time: 0.03 sec

Case 2: Large Input (10,000,000 elements)

- Normal Merge Sort Time: 2.556 sec
- Parallel Merge Sort Time: 1.610 sec

6. Analysis

- For small inputs:
 - Parallel processing is slower due to:
 - Process creation overhead
 - Data transfer cost
- For large inputs:
 - Parallel processing becomes faster
 - Multiple CPU cores are utilized

7. Conclusion

Parallel merge sort improves performance for large datasets by utilizing multiple CPU cores. However, for smaller inputs, sequential merge sort performs better due to lower overhead.